

# LogSight: Adaptive Multi-View Log Anomaly Detection and Root-Cause Localization

Karthick S  
Artificial Intelligence and Data Science  
Rajalakshmi Engineering College Chennai,  
India  
[h2karthi04@gmail.com](mailto:h2karthi04@gmail.com)

Jayasri Archana Devi M.E.,  
Assistant Professor (SG)  
Artificial Intelligence and Data Science  
Rajalakshmi Engineering College Chennai,  
India  
[jayasriarchanadevi.p@rajalakshmi.edu.in](mailto:jayasriarchanadevi.p@rajalakshmi.edu.in)

Koushal V  
Artificial Intelligence and Data Science  
Rajalakshmi Engineering College Chennai,  
India  
[koushaljavanthi@gmail.com](mailto:koushaljavanthi@gmail.com)

Jegadeeswaran D  
Artificial Intelligence and Data Science  
Rajalakshmi Engineering College Chennai,  
India  
[adjujegg@gmail.com](mailto:adjujegg@gmail.com)

**Abstract—** *Large computing systems write millions of log lines, and failures hide among them as unusual messages, rare event types, or abnormal bursts of events. Labelled failure data is rarely available, and rule-based monitoring catches only anticipated problems. This paper presents LogSight, an unsupervised multi-view system that detects anomalies, explains each detection, groups detections into incidents, and ranks likely origin components. Each line is scored through three views: a semantic view (Sentence-BERT embeddings compared with KMeans prototypes), a structural view (level, component, parameters and template rarity, scored by Isolation Forest and Local Outlier Factor) and a temporal view (event order scored by a Hidden Markov Model and event frequency by a rolling z-score). Per-line reliability weights, scaled by each view's informativeness on the unlabelled learning window, fuse the scores, and a rolling median and MAD threshold flags anomalies and grades their severity. A Kolmogorov-Smirnov test with a shuffle control monitors drift. On the BlueGene/L log of 4,713,483 cleaned lines, the pipeline reaches an AUC-ROC of 0.7237 and an AUC-PR of 0.1900 on a chronological test split of 942,697 lines, up from 0.456 once a data-leakage bug and a weak view are corrected. Equal-weight fusion (0.470) falls below the best single view (0.830), and weighted fusion recovers most of this gap. On test lines with templates unseen in training, the AUC-ROC is 0.658. In the FastAPI and React application, per-line embeddings and label-free view weighting raise the semantic view's fusion weight from 11.8% to 38.2% without labels.*

**Keywords—** *log anomaly detection, unsupervised learning, multi-view learning, Sentence-BERT, Isolation Forest, Hidden Markov Model, score fusion, root-cause localization, explainability*

## I. INTRODUCTION

Computing systems record their activity in logs. Each line carries a timestamp, the component that wrote it, a severity level and a free-text message. When something fails, logs are usually the first evidence and sometimes the only one. The BGL log studied here has 4,747,963 raw lines collected over about seven months, which is far too large to read by hand. Deciding which lines matter, why they matter, and where a problem began therefore needs automation.

Log anomalies come in three forms. A semantic anomaly is a message whose meaning is unusual. A structural anomaly is a rare event type or an unusual combination of level, component and parameters. A temporal anomaly is a burst, a repeat, or a rare ordering of events. Logs are also strongly bursty. In BGL, half of all consecutive lines are less than 26 ms apart, so failures tend to appear as dense bursts rather than isolated lines [1]. Most existing detectors target only one of these forms.

Supervised detectors perform well when labels exist, but labelled failures are expensive to obtain and cannot cover failures that have not yet occurred [2]. Count-based statistical methods ignore message meaning and fine timing [3]. Deep sequence models rely on a single view and explain little about individual lines [4]. Root-cause tools usually need dependency graphs or tracing, which many systems, including BGL, do not provide. The unsupervised semantic approach of LogAnomaly [5] is the closest starting point for this work.

This paper makes five contributions: (1) an unsupervised detector that scores every log line from three views, each with a suitable detector; (2) a reliability-weighted fusion scheme in which each view's weight is set per line from its reliability and scaled by its informativeness on the unlabelled learning window; (3) a rolling median and MAD threshold with severity grading, and drift monitoring with a shuffle control; (4) per-line evidence, incident grouping, and correlation-based root-cause ranking that need no dependency information; and (5) a chronological evaluation with an ablation study and an unseen-template test, together with a working application.

## II. RELATED WORK

Parsing and datasets. Drain is an online parser that routes each message through a fixed-depth tree and assigns it to the most similar log group, which turns variable messages into event templates [6]. LogSight uses Drain3, its maintained implementation, and feeds messages in chronological order because the parser is stateful. Loghub collects public system-log datasets, including BGL [7].

Statistical and sequence-based detection. Xu et al. applied PCA to message-count vectors from parsed console logs [3]. Lou et al. mined linear invariants among event counts and treated violations as anomalies [8]. Both work on sessions or windows rather than individual lines. A comparison of supervised and unsupervised methods found that supervised models do better when labels exist, which are rarely available in practice [2]. DeepLog trains an LSTM to predict the next template key and flags unexpected keys [4]. It shows that event order is informative, but it uses a single view.

Semantic and transformer-based methods. LogAnomaly represents templates with semantic vectors so that unseen templates can be matched to similar known ones [5]. LogRobust combines semantic vectors with an attention-based Bi-LSTM and stays robust when log statements change, but it is supervised [9]. Sentence-BERT fine-tunes BERT in siamese networks so that sentence embeddings can be compared by cosine similarity [10]. LogBERT trains BERT on log-key sequences with masked key prediction, which

captures rich context but requires heavy training and gives little per-line explanation [11]. SimCSE improves embeddings through contrastive learning [12]. Hu et al. and Zhang et al. applied Sentence-BERT to log anomaly detection and confirmed that such embeddings suit log messages [13], [14]. LogSight uses the embedding as one of three views rather than as the only signal.

**Outlier detection and sequence likelihood.** Isolation Forest isolates anomalies in fewer random partitions than normal points, giving a fast global score [15]. Local Outlier Factor compares the density of a point with that of its neighbours and finds local outliers [16]. The two are complementary, and together they form the structural view. Hidden Markov Models give the likelihood of a sequence through the forward algorithm [17]. A low likelihood on a window of template identifiers signals an unusual event order that line-level detectors cannot see.

**Robust thresholding and drift.** Leys et al. showed that the mean and standard deviation are distorted by the outliers they are meant to find, and recommended the median and median absolute deviation (MAD) [18]. Gama et al. survey how data distributions change over time [19]. The two-sample Kolmogorov-Smirnov (KS) test compares two distributions without assuming their form [20]. A unified model for several log sources has also been proposed [21]; LogSight is evaluated on BGL only.

**Research gap.** Relative to LogAnomaly [5] and the other works above, five gaps remain: detection from a single view, fixed or uncalibrated score combination, fixed or non-robust thresholds, detection that is separate from explanation and root cause, and drift tests without controls. On bursty logs, consecutive chunks can look different even when nothing has changed, so a drift test needs a control. LogSight does not replace the algorithms reviewed here. It combines them in one unsupervised pipeline that addresses these five gaps.

### III. PROPOSED METHOD

LogSight is a batch system with five layers: ingestion and parsing, multi-view detection, decision, explanation and localization, and storage and presentation. Fig. 1 shows the architecture.

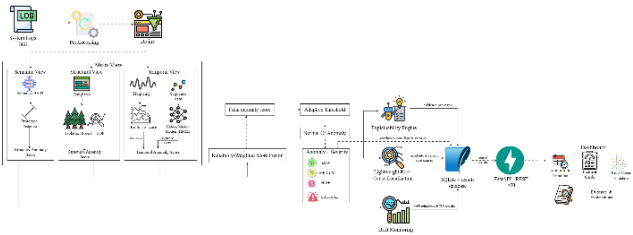


Fig. 1: System architecture of LogSight

#### A. Data Preparation

Each raw BGL line is split into ten fields on runs of whitespace, because splitting on a single space shifts columns where the file contains double spaces. Lines with fewer than ten fields (34,470 lines with no message) are skipped when the file is read. Rows with missing timestamp or time are then removed, along with ten rows whose level field held stray values, leaving 4,713,483 lines. No labelled anomaly is lost in these steps. Lines are sorted by the microsecond time field, since the Unix timestamp has only second precision and

would scramble events inside a burst. Drain3 then mines templates from the message content in chronological order.

The data is split chronologically into 60%, 20% and 20% parts. The first 60% is the learning window, which teaches the detectors what normal behaviour looks like. The split is never random, because a random split would let detectors learn from the future. The learning window is not cleaned of anomalies, since that would require labels, so the detectors must tolerate small contamination. Labels are used by the evaluation scripts and, in v1, to choose  $q$  and  $\lambda$ ; no detector is trained on them, and the application (v2) never reads them.

#### B. Multi-View Scoring

**Semantic view.** Each distinct line text is embedded once with the Sentence-BERT model all-MiniLM-L6-v2, which produces 384-dimensional vectors [10]. MiniBatchKMeans fitted on learning-window embeddings yields 30 prototypes  $c_k$ . For a line with embedding  $e_i$ , the semantic score is

$$s_{\text{sem}}(i) = 1 - \max_k \cos(e_i, c_k). \quad (1)$$

Two embedding granularities were built. The research version (v1) uses one vector per template. The application version (v2) uses one vector per distinct line text, so two lines of the same template can differ. Above 200,000 distinct texts, v2 falls back to one vector per template.

**Structural view.** The input features are the parameter count, level rank, one-hot component and type, and the template frequency counted on the learning window only. The template identifier is kept with each line but is not given to the detectors. Isolation Forest (100 trees) and LOF (20 neighbours, novelty mode, fitted on a 30,000-line learning sample) each produce an outlier score. With  $a_i$  and  $b_i$  the two scores and  $\text{mm}(\cdot)$  min-max scaling,

$$s_{\text{str}}(i) = \frac{1}{2} [\text{mm}(a_i) + \text{mm}(b_i)]. \quad (2)$$

**Temporal view.** Seven timing features are computed from past data only, with no look-ahead: time since the previous line, lines in the last 60 seconds, burst rate, template novelty, causal transition probability, rolling entropy and repeated-event ratio. The temporal score itself uses two signals: the template sequence and the 60-second event frequency. The other features feed severity (repeated-event ratio), reliability and the per-line explanations. A 10-state categorical HMM is fitted on the learning-window template sequence. It scores 20-line windows every 10 lines, with the per-line negative log-likelihood  $h_i$  forward-filled between windows. A rolling z-score  $z_i$  (window 50) is computed on the 60-second event frequency. The temporal score is

$$s_{\text{tem}}(i) = \frac{1}{2} [\text{mm}(h_i) + \text{mm}(|z_i|)]. \quad (3)$$

In the application, a window containing a template that the HMM never saw in training receives the highest finite score observed so far.

#### C. Reliability-Weighted Fusion

Each view  $v$  has a reliability  $r_{i,v}$  on line  $i$ . For the semantic view it is the mean cosine similarity to the 10 nearest prototypes. For the structural view it is the log-scaled, normalised template frequency. For the temporal view it is the inverse of the rolling variance of event frequency over a window of 100 lines. The weight of view  $v$  on line  $i$  is

$$w_{i,v} = \frac{q_v e^{r_{i,v}}}{\sum_u q_u e^{r_{i,u}}}, \quad (4)$$

where  $q_v$  is the quality of view  $v$ . This is one softmax over  $r_{i,v} + \ln q_v$ , so the weights on each line sum to 1. The fused score is

$$S_i = \sum_v w_{i,v} s_v(i). \quad (5)$$

In v1, fixed quality multipliers of 0.2, 1.0 and 0.5 (semantic, structural, temporal) were used after per-view AUC was measured on the labelled test split. In v2, quality is measured without labels as the spread between the 99th and 50th percentiles of a view's scores on the learning window, divided by the largest such spread, with a floor of 0.1.

#### D. Adaptive Threshold and Severity

Line  $i$  is flagged when  $S_i$  exceeds a rolling threshold

$$\tau_i = \text{median}_{200}(S) + \lambda \cdot 1.4826 \cdot \text{MAD}_{200}(S), \quad (6)$$

computed over a window of 200 lines with  $\lambda = 1.5$ . The constant 1.4826 scales the MAD to match the standard deviation under normality. The median and MAD change little when a few anomalies fall inside the window, unlike the mean and standard deviation [18].

Flagged lines receive a severity score

$$\text{sev}_i = 0.4 S_i + 0.25 \text{persistence}_i + 0.2 \text{freq}_i + 0.15 \text{rarity}_i. \quad (7)$$

The application grades severity by quantiles among flagged lines: the bottom 50% are Low, 50 to 85% Medium, 85 to 98% High, and the top 2% Critical.

#### E. Evidence, Drift and Root-Cause Localization

**Evidence.** For the 200 most severe flags in a run, an evidence package records the line, component, node and template, the final score and severity, each view's score and weight, and the dominant view (largest score times weight). It also records the nearest normal line of the same template by cosine similarity, and the five preceding events. Because the fused score is a weighted sum, each view's contribution is exactly  $\phi_v = w_v (s_v - \bar{s}_v)$ , where  $\bar{s}_v$  is the mean of  $s_v$  over non-flagged lines. With the line's weights held fixed, the base value  $B = \sum_v w_v \bar{s}_v$  plus the three contributions equals the final score, and the dashboard shows this as a Shapley-style waterfall.

**Drift.** Two signals are monitored: the embedding distance from the learning-window centroid, and template frequency. Each 50,000-line window is compared with a 50,000-value learning-window sample by a two-sample KS test. Drift is reported after three consecutive windows with  $p < 0.01$ . Sub-sampling is needed because a test against 2.8 million reference values makes trivial differences look significant. A

shuffle control splits the learning window randomly and tests the two halves against each other. A near-zero statistic confirms that the computation is correct and that any sequential drift signal comes from temporal clustering.

**Root cause.** Flags within 10 minutes of each other are chained into one incident. Component pairs that appear in the same incident are counted as co-occurrence. Each component  $c$  of an incident is ranked by  $\text{score}_c = 0.35 P_c + 0.30 V_c + 0.20 F_c + 0.15 C_c$ , where  $P_c = 1/(1 + t_c)$  is first-occurrence priority ( $t_c$  = seconds from the incident start to the first flag of  $c$ ),  $V_c$  average severity,  $F_c$  in-incident frequency and  $C_c$  co-occurrence centrality, each min-max normalised within the incident. The result is a ranked list of candidates based on correlation. It is not a proven cause, because BGL provides no service-dependency information.

#### F. Application Architecture

The pipeline is wrapped in a FastAPI backend with 22 REST endpoints and a nine-table SQLite database that stores runs, events, templates, incidents, root-cause candidates, co-occurrence, drift results and evidence packages. Uploaded logs are processed by a background worker, and each run stores its parameter snapshot and the facts of every fitted detector. A React dashboard reads every number from the API and shows run progress, timelines, incident cards, a root-cause cluster map and per-line attribution. Deployment uses Docker Compose with nginx.

### IV. EXPERIMENTAL SETUP

**Dataset.** The BGL log from Loghub has 4,747,963 lines from 3 June 2005 to 4 January 2006 [7]. Skipping 34,470 lines with no message and 10 malformed rows leaves 4,713,483 lines, of which 348,460 (7.39%) are labelled anomalies. Drain3 produced 1,819 templates, of which 1,161 (63.83%) occur only once. Most singletons are register-dump lines whose hexadecimal values share no structure. Retuning the parser to merge them risked merging genuinely different events, so the template count was kept and the semantic and temporal views cover the gap. Table I summarises the data and split.

Table I. Dataset, Parsing and Split Statistics

| Measure                        | Value                                        |
|--------------------------------|----------------------------------------------|
| <b>Cleaned lines</b>           | 4,713,483                                    |
| <b>Labelled anomalies</b>      | 348,460 (7.39%)                              |
| <b>Templates / singletons</b>  | 1,819 / 1,161 (63.83%)                       |
| <b>Learning window (train)</b> | 2,828,089 lines                              |
| <b>Validation</b>              | 942,697 lines                                |
| <b>Test</b>                    | 942,697 lines (46,283 anomalies, about 4.9%) |

**Software.** The implementation uses Python 3.10, Drain3 0.9.11, sentence-transformers, scikit-learn 1.7.2 and hmmlearn 0.3.3. The backend uses FastAPI and SQLAlchemy, and the frontend uses React 19 with TypeScript.

**Protocol.** Metrics are computed on the test split. Precision, recall and F1 are computed on the flags, and AUC-ROC and

AUC-PR on the final score. Accuracy is not reported, because flagging nothing would already give about 93% at a 7.39% anomaly rate. The ablation averages the views of each configuration with equal weights under the same threshold. The unseen-template test evaluates test lines whose template never occurs in the learning window separately from lines with seen templates. The research numbers describe v1 on the full log, and the application (v2) is demonstrated on five continuous 50,000-line samples (Table II).

Table II. Research Pipeline (v1) Versus Application (v2)

|                  | Research (v1)               | Application (v2)     |
|------------------|-----------------------------|----------------------|
| <b>Data</b>      | Full BGL                    | 50,000-line samples  |
| <b>Embedding</b> | Per template                | Per distinct line    |
| <b>Quality q</b> | 0.2 / 1.0 / 0.5 (test AUC)  | P99 – P50, no labels |
| <b>Labels</b>    | Evaluation; q and $\lambda$ | Never read           |

## V. RESULTS AND DISCUSSION

### A. Detection Performance

The first fused score gave an AUC-ROC of 0.456 on the test split, which is worse than chance. The per-view values were 0.306 (semantic), 0.547 (structural) and 0.428 (temporal). The semantic view was anti-correlated with anomalies. With one embedding per template, every line of a template has an identical vector, so an anomalous instance of a common template cannot be told apart from its normal instances.

Two corrections followed. Applying view-quality multipliers raised the fused AUC-ROC to 0.554. A data-leakage bug was then found: the template-frequency feature had been counted over the whole log, including the test period. Counting it on the learning window only raised the fused AUC-ROC to 0.7237. The fix changed only the structural view, whose AUC-ROC rose from 0.547 to 0.830. Table III shows the progression.

Table III. Auc-Roc Progression Across Corrections

| Stage                                     | Semantic | Structural | Temporal | Fused  |
|-------------------------------------------|----------|------------|----------|--------|
| <b>Initial fused score</b>                | 0.306    | 0.547      | 0.428    | 0.456  |
| <b>+ view-quality multipliers</b>         | 0.306    | 0.547      | 0.428    | 0.554  |
| <b>+ frequency leakage fix (final v1)</b> | 0.306    | 0.830      | 0.428    | 0.7237 |

The final v1 test metrics at  $\lambda = 1.5$  are a precision of 0.0806, a recall of 0.1625 and an F1 of 0.1078, with 93,244 of 942,697 lines flagged. The AUC-ROC is 0.7237 and the AUC-PR is 0.1900. A random score would reach an AUC-PR equal to the test anomaly rate of about 4.9%, so the result is about 3.9 times the random baseline. The precision is low

in absolute terms, and the flag-level metrics should be read together with the threshold-independent AUC values.

### B. Ablation Study

Table IV. Ablation results (equal-weight averaging)

| Configuration                    | AUC-ROC | AUC-PR |
|----------------------------------|---------|--------|
| <b>Semantic only</b>             | 0.306   | 0.063  |
| <b>Structural only</b>           | 0.830   | 0.657  |
| <b>Temporal only</b>             | 0.428   | 0.052  |
| <b>Semantic + structural</b>     | 0.465   | 0.067  |
| <b>All three (equal weights)</b> | 0.470   | 0.048  |

Averaging all three views equally (0.470) is far worse than the structural view alone (0.830), because the two weak views drag down a strong one. The weighted v1 pipeline (0.7237, with quality multipliers) recovers most of that gap, which shows that weak views must be down-weighted. The structural view alone still beats the tuned v1 pipeline. This motivated the label-free view quality of v2, and it remains an open point.

### C. Unseen Templates

Table V. Unseen versus seen templates on the test split

| Subset                  | Lines   | Anomalies | Base rate | AUC-ROC | AUC-PR | AUC-PR / base rate |
|-------------------------|---------|-----------|-----------|---------|--------|--------------------|
| <b>Unseen templates</b> | 749,626 | 43,679    | 5.83 %    | 0.658   | 0.252  | 4.3x               |
| <b>Seen templates</b>   | 193,071 | 2,604     | 1.35 %    | 0.936   | 0.103  | 7.7x               |
| <b>All test lines</b>   | 942,697 | 46,283    | 4.91 %    | 0.724   | 0.190  | 3.9x               |

Three points stand out. First, 79.5% of test lines carry templates that never appear in the learning window, so unseen patterns are the common case in BGL and not an edge case. Second, seen templates rank anomalies very well (AUC-ROC 0.936), but precision there is only 0.029 because anomalies are rare among them. Third, unseen templates have the higher raw AUC-PR mainly because anomalies are about four times more common there. Relative to the base rate, seen templates do better (7.7 times against 4.3 times), which matches their higher AUC-ROC.

### D. Threshold and Drift

$\lambda$  was swept over {1.0, 1.5, 2.0, 2.5, 3.0, 4.0} on the test split. With  $\lambda = 1.0$ , F1 was highest (0.1117) but about 27% of the test set was flagged. With  $\lambda = 1.5$ , F1 was 0.1078 and about 10% was flagged. The larger value keeps almost the same F1 at a flag volume that is far more practical for human review.

Sequential KS windows flagged drift at row 100,000, which lies inside the learning window itself. The shuffle control on the same data showed no difference (KS about

0.001,  $p > 0.3$  for both signals). The computation is therefore correct, and the flag reflects the fact that BGL's dominant templates occur in time-concentrated bursts, which makes any two consecutive chunks look different. On BGL the sequential flags should be read as a demonstration of the method, not as evidence of real drift. This is the reason a control is needed.

#### E. Application Results

The application was run on BGL sample 1 (50,000 lines, 16 to 21 June 2005). Two changes separate v1 from v2 in the semantic view: one embedding per distinct line instead of per template, and label-free view quality in place of the fixed 0.2 multiplier. Together they changed the semantic view from inactive to influential (Table VI). Its mean fusion weight rose from 11.8% to 38.2% and the number of flags it drove rose from 0 to 3,208, without using any labels. Embedding the roughly 21,000 distinct lines takes about 40 seconds on a CPU. Total flags rose by about 15%, from 7,779 to 8,980. This shows that the view is active, but it does not show that the view is more accurate, because the application reads no labels. This shows that the view is active, but it does not show that the view is more accurate, because the application reads no labels.

Table VI. Semantic view v1 versus v2 (BGL sample 1)

| Measure                                         | v1                    | v2                    |
|-------------------------------------------------|-----------------------|-----------------------|
| Mean weight (semantic / structural / temporal)  | 11.8% / 69.3% / 19.0% | 38.2% / 40.7% / 21.1% |
| Flags driven (semantic / structural / temporal) | 0 / 3,294 / 4,485     | 3,208 / 2,065 / 3,707 |
| Distinct semantic scores                        | 192                   | 21,111                |

The current application also uses a revised HMM rule for unseen templates and quantile severity buckets, so a run of it on the same sample gives slightly different totals from Table VI. The run found 194 templates, 8,998 flagged lines (18.0%) and 96 incidents. The severity split was 180 Critical (2%), 1,170 High (13%), 3,149 Medium (35%) and 4,499 Low (50%). The flag rate is above the labelled rate of 6.2% because a rolling MAD threshold always flags the upper tail of each window. On synthetic noise with the same window and  $\lambda$ , it flagged 0.4% of uniform noise, 6.9% of normal noise and 19.7% of log-normal noise. Part of the flag rate therefore depends on the shape of the score distribution and not only on real anomalies. Severity grading and incident grouping help reviewers focus on the most important flags.

The backend test suite of 28 automated tests passes. It covers the API endpoints, configuration handling, a 4,000-line end-to-end run in which every detector is fitted on that upload's learning window, and detection unit checks. A separate script re-derives every score from the fitted detectors and confirms that the fusion weights sum to 1 on every line.

#### F. Dashboard

The overview page shows headline counts, the anomaly timeline against the moving cutoff, and the severity distribution. The incidents page (Fig. 2) shows a two-dimensional cluster map in which each circle is a likely-origin component sized by incident count, with its largest incidents as dots. The flagged-line page (Fig. 3) shows the

evidence package and the exact attribution of the final score to the three views.

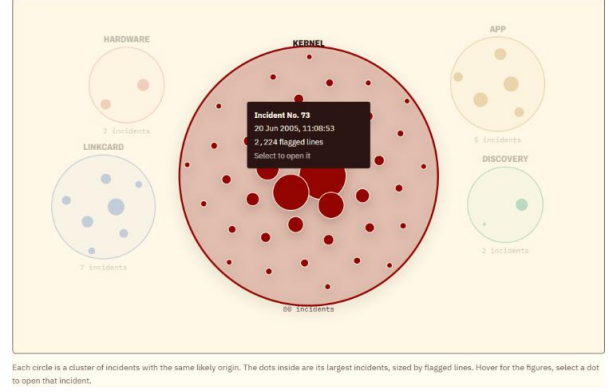


Fig. 2: Incidents page, 2D root-cause cluster map

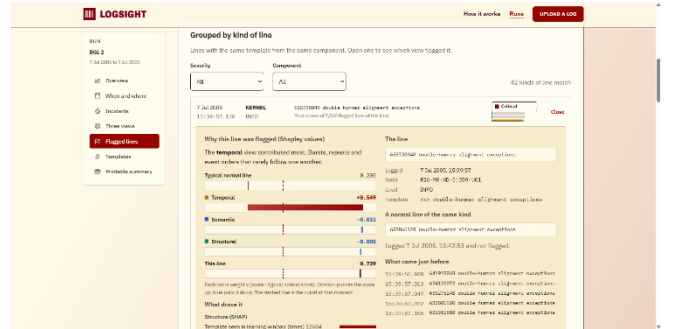


Fig. 3: Flagged line detail with Shapley attribution

#### G. Capability Comparison

Table VII compares capabilities with existing approaches. No numeric comparison with published results is made, because the reviewed works use different preprocessing, session or sliding-window formulations instead of per-line scoring, and different splits.

Table VII. Feature comparison with existing approaches

| Capability                     | Rule-based | PCA / invariants | Deep Log | LogRobust | LogBERT | LogSight |
|--------------------------------|------------|------------------|----------|-----------|---------|----------|
| Works without labels           | Yes        | Yes              | Yes      | No        | Yes     | Yes      |
| Semantic view                  | No         | No               | No       | Yes       | No      | Yes      |
| Structural view                | Partly     | Yes              | No       | No        | No      | Yes      |
| Temporal / sequence view       | Partly     | No               | Yes      | Yes       | Yes     | Yes      |
| Per-line reliability weighting | No         | No               | No       | No        | No      | Yes      |
| Adaptive robust                | No         | No               | No       | No        | No      | Yes      |

| threshold             |           |         |              |           |         |     |
|-----------------------|-----------|---------|--------------|-----------|---------|-----|
| Per-line evidence     | Rule text | Limited | Limited      | Attention | Limited | Yes |
| Root-cause candidates | No        | No      | Workflow     | No        | No      | Yes |
| Drift monitoring      | No        | No      | Model update | No        | No      | Yes |

### H. Insights

Four lessons stand out. Adding views can hurt unless they are weighted by reliability and quality. A single feature computed over the whole log was the largest source of error, and fixing that leak gave the largest AUC gain. Design choices in the semantic view have measurable effects: per-template embeddings made the view useless within a template, while per-line embeddings, together with label-free view weighting, made it the second most influential view. Finally, drift tests on bursty logs need a control, because without the shuffle control the temporal clustering in BGL would have been mistaken for drift.

## VI. LIMITATIONS

The system currently supports only the BGL format, and processing is batch only. The full 4.7-million-line log takes hours and several GB of memory. The headline metrics come from v1, and v2 has not been evaluated at full scale. The value of  $\lambda$  and the v1 quality multipliers were chosen by looking at test-split results, and the leakage bug was also found through test AUC. The 20% validation split was created but not used, so the reported numbers may be optimistic and validation-based tuning is needed. In v2, severity buckets are relative, so about 2% of flags are always Critical. The first-occurrence priority in the root-cause ranking decays very quickly and can dominate the ranking. The 10-minute incident gap can split long cascades. Root causes are correlation-based candidates and not proof.

## VII. CONCLUSION AND FUTURE WORK

This paper presented LogSight, an unsupervised multi-view system for log anomaly detection and root-cause localization. Each line is scored for meaning, structure and timing. The scores are fused by per-line reliability, flagged by a robust adaptive threshold and graded by severity. Flags are explained with evidence and grouped into incidents with ranked root-cause candidates, and drift is checked with a KS test and a shuffle control. On the full BGL log, the research pipeline reached an AUC-ROC of 0.7237 and an AUC-PR of 0.1900 on a chronological split. It stayed above chance (0.658) on the nearly 80% of test lines with unseen templates. The work was delivered as a full-stack application with a React dashboard and Docker deployment. Future work includes a full-scale evaluation of v2 and validation-based tuning of key parameters. It also includes improving the root-cause ranking, adding parsers for HDFS and Thunderbird, adding a streaming mode with drift-

triggered retraining, and contrastive fine-tuning of the embedding model [12].

## REFERENCES

- [1] A. J. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, Edinburgh, U.K., 2007, pp. 575-584.
- [2] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proc. IEEE 27th Int. Symp. Software Reliability Engineering (ISSRE)*, Ottawa, ON, Canada, 2016, pp. 207-218.
- [3] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. ACM SOSP*, Big Sky, MT, USA, 2009, pp. 117-132.
- [4] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Dallas, TX, USA, 2017, pp. 1285-1298.
- [5] W. Meng et al., "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Macao, China, 2019, pp. 4739-4745.
- [6] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Honolulu, HI, USA, 2017, pp. 33-40.
- [7] S. He, J. Zhu, P. He, and M. R. Lyu, "Loghub: A large collection of system log datasets towards automated log analytics," arXiv:2008.06448, 2020.
- [8] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li, "Mining invariants from console logs for system problem detection," in *Proc. USENIX Annu. Technical Conf.*, Boston, MA, USA, 2010, pp. 231-244.
- [9] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM ESEC/FSE*, Tallinn, Estonia, 2019, pp. 807-817.
- [10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, Hong Kong, China, 2019, pp. 3982-3992.
- [11] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, Shenzhen, China, 2021, pp. 1-8.
- [12] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," in *Proc. EMNLP*, Online and Punta Cana, Dominican Republic, 2021, pp. 6894-6910.
- [13] C. Hu, X. Sun, H. Dai, H. Zhang, and H. Liu, "Research on log anomaly detection based on Sentence-BERT," *Electronics*, vol. 12, no. 17, p. 3580, 2023.
- [14] M. Zhang, J. Chen, J. Liu, J. Wang, R. Shi, and H. Sheng, "LogST: Log semi-supervised anomaly detection based on Sentence-BERT," in *Proc. 7th Int. Conf. Signal and Image Processing (ICSIP)*, Suzhou, China, 2022, pp. 356-361.
- [15] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413-422.
- [16] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Dallas, TX, USA, 2000, pp. 93-104.
- [17] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proc. IEEE*, vol. 77, no. 2, pp. 257-286, Feb. 1989.
- [18] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *J. Exp. Social Psychology*, vol. 49, no. 4, pp. 764-766, Jul. 2013.
- [19] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, Art. no. 44, pp. 1-37, Mar. 2014.
- [20] F. J. Massey, "The Kolmogorov-Smirnov test for goodness of fit," *J. Amer. Statistical Assoc.*, vol. 46, no. 253, pp. 68-78, Mar. 1951.
- [21] R. Zang et al., "MLAD: A unified model for multi-system log anomaly detection," arXiv:2401.07655, 2024.